

Considerations for the design of expressive portable SIMD vectors

Doc. No.:	P0203R0
Date:	2016-01-26
Project:	Programming Language C++, SG14 Low Latency
Reply To:	Mathias Gaunard < mgaunard@bloomberg.net >

Table of Contents:

- [Introduction](#)
- [Background](#)
- [Number of elements in a SIMD vector](#)
 - [Relationship between vectors of different types](#)
 - [User-defined number of elements of the SIMD vector](#)
- [Missing functionality](#)
 - [Combining operations](#)
 - [Promotion and conversion operations](#)
 - [Shuffle operations](#)
- [Implementation concerns](#)
 - [Aliasing](#)
 - [Size of SIMD vector object](#)
 - [Impact of calling conventions](#)
- [Syntax](#)
 - [Mask and bools](#)
 - [Expression templates for syntactic sugar](#)
- [References](#)

Introduction

Single Instruction, Multiple Data (SIMD) is a class of computers that operate the same operation on multiple elements of data in parallel, akin to vector processing. Modern computers provide SIMD instructions that operate on special registers representing short vectors, typically of a few words in size, and the process of converting normal control flow to SIMD control flow is referred to as vectorization.

Two approaches have been considered to provide C++ programmers with the ability to make use of Single Instruction, Multiple Data (SIMD) architectures:

- annotations on loops to direct automatic vectorization.
- explicit usage of short vectors within the type system.

This paper covers the second approach exclusively. The subject has already been covered in previous papers: [N3571](#), [N4184](#) and a number of supporting papers.

The considerations in this paper cover how to design the API of SIMD vectors so that the provided programming paradigm can be sufficiently expressive to address many applications while also taking into account the variability of hardware support.

Background

The author of this paper is one of the authors of the original SIMD proposal [N3571](#) as well as the Boost.SIMD (not a Boost library, candidate for inclusion into Boost) and NT² libraries.

Particularly of interest to the committee, the Boost.SIMD library, through its various authors, has explored the feasibility of implementing all C++ transcendental functions on SIMD types, portably across several hardware platforms with a minimal abstraction layer. This has led to some insight as to what is required from the API and what were in retrospect bad design decisions.

Number of elements in a SIMD vector

Relationship between vectors of different types

SIMD registers of, say, 128 bits, are typically able to contain 4 32-bit values or 2 64-bit values; the same can be generalized to SIMD registers of various bit-widths `width` with the relationship that for any arithmetic types `T` and `U` ,

```
vector<T>::static_size * sizeof(T) == vector<U>::static_size * sizeof(U) == width/CHAR_BIT .
```

This relationship is useful and programmers will typically rely on it when promoting smaller types to larger ones: promoting a vector of 4 single-precision floats will give 2 vectors of 2 doubles each. Example uses of that technique is for mixed-precision programming, where part of a double-precision computation will be done using single-precision to quickly compute an approximation using the augmented parallelization, then refining it back with double precision. Signal processing is somewhat similar: you might have to promote a 8-bit integer stream to 16-bit during computation to avoid overflows.

One may also want to work together with both floating-point and integers vectors of relatable sizes when, for example, manipulating the mantissa or exponent of the floating-point directly, as is required to achieve sufficient accuracy for a number of algorithms, like exponentiation.

However, there are good reasons why an implementation may choose not to satisfy this property: some architectures provide SIMD instructions for some types, but not others. (older AltiVec and NEON don't have double precision, AVX pre-AVX2 doesn't have integers, etc.)

Conclusion: to write portable code, it is either required that vectors for all types use the same register width, or that the user may choose himself the number of elements of the vectors.

Suggestion: don't specify any relationship between vectors for different types, but allow the default size to be overridable.

User-defined number of elements of the SIMD vector

In the latest proposal, the number of elements of a SIMD vector is implementation-defined to the most appropriate one for the target architecture, leading to variations across implementations.

The author of [N4184](#) made the claim that all code could, and should, be written in a way that is agnostic of the number of elements, demonstrating for example how to achieve this with general matrix-matrix multiplication [N4454](#), and that there is therefore no need to provide tools to cater to bad programming practices.

Generalization is not always easy

The author of this paper believes this lacks practicality and might be too idealistic; most existing code today using explicit SIMD vectors, regardless of programming language, is programmed with a specific size in mind and cannot easily be generalized to an arbitrary number of elements. Some algorithms also just cannot necessarily be expressed in a size-agnostic away, at least not without heavy meta-programming: the Fast Fourier Transform butterfly schemes¹ or the bitonic merge networks of AA-sort², that both require shuffling around a number of values across vectors, are a good example of this. Even some matrix multiplication algorithms, like those used in state-of-the-art libraries, use intra-register operations to transpose values and cannot be written as simply as in [N4454](#).

Data can be small and not chunkable

Sometimes, one only has a given small number of values for which they'd like to operate on parallel. While one may argue that this is a design problem and memory should be rearranged in a more structure-of-arrays layout, it might be that it not possible due to memory constraints or that the problem does not offer other dimensions with more parallization potential anyway.

Sized vectors give a better API when multiple types are involved

Finally, enabling explicit number of elements allow to provide clearer and more high-level interfaces when dealing with inputs of different types.

```
simd_vector<double, N> ldexp(simd_vector<double, N> x, simd_vector<int, N> exp);
```

is a fairly natural declaration of the standard `double ldexp(double x, int exp);` function extended to SIMD.

However, assuming that a double is 64-bit and an int 32-bit, this means that `x` and `exp` would not have the same number of fixed-width SIMD registers.

The alternatives, which would only work if all vectors used the same SIMD register width, would be as follows:

```
// only read low half of `exp`
simd_vector<double> ldexp(simd_vector<double> x, simd_vector<int> exp);

// as many values for both `x` and `exp`
array<simd_vector<double>, 2> ldexp(array<simd_vector<double>, 2> x, simd_vector<int> exp);
```

Other problems with this approach for example is when computation requires promotion, you end up with twice as many variables, and

need to repeat computations on each of them, which is not a nice programming model.

Sized vectors better address architectures with multiples widths

Some architectures like NEON also have two vector sizes: 64-bit and 128-bit, and depending of which one you use you halve the number of registers available in half. Some instructions will only be defined on the smaller or larger vectors, for valid reasons (ex: full multiplication: $\text{int32x2} * \text{int32x2} \rightarrow \text{int64x2}$; promotion: $\text{int32x2} \rightarrow \text{int64x2}$)

Multiple register sizes need to be able to exist in the same program to make the most of those architectures; forcing a single size would ignore more than half of the instruction set.

Conclusion: being able to specify the size explicitly is useful for applications that are specific to a given size and also gives a nice API to work with, even if that means that a vector can actually be represented by multiple registers

Suggestion: make vector be defined as

```
template<class T, class X = /* implementation-defined ABI tag */>
int best_size_v = /* implementation-defined */;

template<class T, int N = best_size_v<T>, class X = /* implementation-defined ABI tag */>
struct simd_vector;
```

with `N` any power of 2 up to a certain value.

Missing functionality

Combining operations

If vectors of arbitrary power-of-2 sizes can be defined, it would be useful to provide functions to combine/slice vectors into larger/smaller ones.

```
template<class T, int N>
simd_vector<T, N*2> combine(simd_vector<T, N> a, simd_vector<T, N> b);

template<class T, int N>
array<simd_vector<T, N/2>, 2> slice(simd_vector<T, N> a);
```

Promotion and conversion operations

Converting between integer and floating-point, and promoting/demoting types are core features that are missing from [N4184](#), and that have important repercussions on the design of the API.

Various options are possible, but the most generic one would be to provide something like

```
vector<T, N> a;
vector<U, N> b = simd_cast<U>(a);
```

Shuffle operations

Shuffle operations, i.e. operations that reorder the elements within a SIMD register, are also missing entirely from [N4184](#). Given that some SIMD architectures have entire units of the hardware dedicating to permuting all values quickly, it seems a shame not to provide access to that functionality.

Anything involving shuffles would typically not be easy to generalize to arbitrary sizes, though a few patterns can be made size-agnostic.

```
template<int... I, class T>
simd_vector<T, sizeof...(I)> shuffle(simd_vector<T, sizeof...(I)> a);
```

Semantics: `simd_vector<T, sizeof...(I)>{ (I < 0 ? 0 : a[I])... };`

```
template<int... I, class T>
simd_vector<T, sizeof...(I)> shuffle(simd_vector<T, sizeof...(I)> a, simd_vector<T, sizeof...(I)> b);
```

Semantics: `slice(shuffle<I..., I...>(combine(a, b)))[0]`

Note: this definition of shuffle is compatible with GCC `__builtin_shuffle`, Clang `__builtin_shufflevector` and OpenCL `shuffle/shuffle2` functions.

Implementation concerns

Aliasing

Traditionally, some vendor-specific APIs, such as the Intel intrinsics, have allowed aliasing between vectors and scalars.

[N4184](#) suggests allowing scalars to alias vectors, but not the other way around. Let's consider both sides.

Vector aliasing scalars

Vectors aliasing scalars is mostly useful to be able to write code like this:

```
void process(float* my_aligned_data)
{
    simd_vector<float>* my_vector_data = reinterpret_cast< simd_vector<float>* >(my_aligned_data);
    // manipulate my_vector_data...
}
```

This allows to treat existing memory as if being vectors without having to go through the overhead of explicit load/store logic. While some users have requested this feature in Boost.SIMD quite adamantly, it seems unnecessary in the presence of iterator or range adaptors.

The main advantage of allowing this is to pass a vector of raw memory by reference with maximum efficiency (without this, some compilers will require copying the memory on the stack).

Scalars aliasing a vector

Scalars aliasing vectors is essentially this

```
simd_vector<float> v;
float* p = &v[0];
p[3] = 42.f;
```

While this gives a nice interface, this actually forces a vector, which is supposed to be in a register, to go into memory, which is generally undesired. For maximum optimization opportunities, it would be best if everything could stay in registers, and only load/store goes to memory.

Moreover, implementing this requires using unions or special compiler attributes like `__may_alias__`; extensive testing of this approach shows that this tends to be fragile with a lot of compiler versions and targets.

Conclusion: not allowing any aliasing gives maximum opportunity for optimization and fits well with the idea that vectors should be registers

Suggestion: make operator[] be read-only and return by value, only allow access with memory through load/store.

Size of SIMD vector object

To satisfy the principle of least surprise, one might argue that `simd_vector<T, N>` should be the same size as `T[N]`, and possibly also compatible layout (see aliasing).

However it appears interesting to let implementations have the freedom to implement say `simd_vector<float, 2>` with the same backend as `simd_vector<float, 4>`.

Suggestion: `sizeof(simd_vector<T, N>) >= sizeof(T) * N`

Impact of calling conventions

Since `simd_vector` is defined as a class, it would use the standard calling convention for user-defined types, which with many ABIs is to pass the value through the stack. Some ABIs also do not allow passing by value such a type due to its alignment requirements, and require passing by const-reference reference instead.

This is undesirable and very negatively affects performance, so much that experience has shown that using attributes such as `__forceinline` or `__attribute__((always_inline))` was necessary to obtain acceptable performance, as the penalty for going through the stack is important and not taken into account by the compiler inliner.

As such, a pure library implementation might be limited and the library might require compiler support so that the `simd_vector` types can be passed to functions with maximum efficiency.

Syntax

Mask and bools

Papers [N4184](#) and [N4185](#) use `Vector<T>` to represent vectors of arithmetic values and `Mask<T>` to represent vectors of boolean values resulting from operations on values of type `Vector<T>`.

It suggests alternatives, like `Vector<bool>`, which isn't sufficient, but it doesn't suggest the following alternative: `Vector<Mask<T>>` or, which a different naming convention, `simd_vector<simd_bool<T>>`. This has the advantage that vectors of boolean results remain vectors and behave similarly to their arithmetic counterparts.

Expression templates for syntactic sugar

Conditionals

Paper [N4185](#) suggests using expression templates to enable the following syntax

```
a(cond) = expr;
```

as a shortcut for

```
a = iif(cond, expr, a);
```

Requiring the use of expression templates is not without problems:

- it makes the return type of expressions not straightforward, results are not of the expected logical type but a proxy
- problems with auto and lifetime of objects
- additional abstraction and indirections which can make it harder for the compiler to optimize (had major problems with this on MSVC)
- bad compatibility with user-written functions

Suggestion: extend the language to provide an overloadable conditional operator so that one could write

```
a = cond ? expr : a;
```

or even

```
a := cond ? expr;
```

Operator dot

[N4184](#) introduces special syntax based overloading `operator[]` to call operator dot for vectors of structures.

Suggestion: build a satisfying way to overload operator dot in the language, see [P0060R0](#).

References

[N3571](#)

A Proposal to add Single Instruction Multiple Data to the Standard Library

[N4184](#)

SIMD Types: The Vector Type & Operations

[N4185](#)

SIMD Types: The Mask Type & Write-Masking

[N4454](#)

SIMD Types Example: Matrix Multiplication

Function Object-Based Overloading of Operator Dot

1. Bader, David A., and Virat Agarwal. "FFTC: fastest Fourier transform for the IBM cell broadband engine." High Performance Computing-HiPC 2007. Springer Berlin Heidelberg, 2007. 172-184.[↩](#)
2. Inoue, Hiroshi, et al. "AA-sort: A new parallel sorting algorithm for multi-core SIMD processors." Proceedings of the 16th International Conference on Parallel Architecture and Compilation Techniques. IEEE Computer Society, 2007.[↩](#)